Lambda, Method References and Stream API (Java 8)

Functional style programming with live demos and diagrams

Agenda

- Why Java 8? Functional interfaces recap
- Lambda expressions: syntax, variants, rules
- Method references: four types
- Stream API: concept, pipeline, class diagram
- Intermediate and terminal operations (filter, map, distinct, limit, sorted)
- Terminal operations: forEach, count, collect, reduce, Optional
- Employee examples: filtering and sorting with streams
- Best practices, mistakes, practice questions

Why Java 8? From Loops to Functions

- Java 8 (2014) added functional-style programming to Java
- Lambda expressions: pass behaviour (code) as a value
- Method references: shorthand for lambdas that call one method
- Stream API: process collections in a declarative pipeline
- Default and static methods in interfaces
- Result: shorter, clearer code - say WHAT you want, not HOW to loop
- Before (Java 7): for (Employee e : el) { if (e.getSalary() > 25000) print(e); }
- After (Java 8): el.stream().filter(e -> e.getSalary() > 25000).forEach(System.out::println);

Functional Interfaces Used in Streams

Interface	Abstract method	Used by	Example from our demos
Predicate<T>	boolean test(T t)	filter	a -> a > 10
Function<T,R>	R apply(T t)	map	str -> str.length()
Consumer<T>	void accept(T t)	forEach	str -> System.out.println(str)
BinaryOperator<T>	T apply(T t, T u)	reduce	(a, b) -> a + b
Supplier<T>	T get()	generators	() -> "Hello"

- All live in java.util.function; a functional interface has exactly ONE abstract method (SAM)

Lambda Expression - Anatomy



- A lambda is an anonymous function: no name, no class, just parameters and a body
- It can be used ONLY where a functional interface is expected
- The target type decides what it means: Calculator cal = (a, b) -> a + b;
- The compiler infers parameter types from the interface method: double add(double a, double b)

Anonymous Inner Class vs Lambda

```
// Java 7: anonymous inner class (lots of boilerplate)
SayHello s1 = new SayHello() {
    @Override
    public void say() {
        System.out.println("Hello! Welcome ");
    }
};

// Java 8: lambda expression (same behaviour, one line)
SayHello s2 = () -> System.out.println("Hello! Welcome ");

s1.say();
s2.say();
```

Key points

- Only the method body really matters; the rest is ceremony
- Lambda replaces an anonymous class for functional interfaces
- Both print: Hello! Welcome

Lambda Syntax Variants

Form	Example	Notes
No parameter	() -> System.out.println("Hello! Welcome ")	empty brackets are required
One parameter	str -> System.out.println(str)	type inferred, brackets optional
With type	(String a) -> a.length()	explicit parameter type
Many parameters	(a, b) -> a + b	brackets required
Block body	(String a) -> { int len = a.length(); return len; }	needs return and ;

- Expression body returns its value automatically; block body must use return when a value is expected

LambdaDemo: The Interfaces

```
@FunctionalInterface
interface SayHello {
    public void say();
    // public void show();    // ERROR: second abstract method
}

interface PrintMsg {
    public void print(String str);
}
interface LengthFinder {
    public int findLength(String s);
}
interface Calculator {
    public double add(double a, double b);
}
```

Key points

- Each interface has exactly ONE abstract method (SAM)
- @FunctionalInterface asks the compiler to enforce that rule
- PrintMsg has no annotation but is still functional
- Return type and parameters decide the lambda's shape

LambdaDemo: Using the Lambdas

```
SayHello s = () -> System.out.println("Hello! Welcome ");
s.say();

PrintMsg p = str -> System.out.println(str);
p.print("UCOE");

LengthFinder l = a -> a.length();
int le = l.findLength("Universal College of Engineering");
System.out.println(le);

Calculator cal = (a, b) -> a + b;
double res = cal.add(100, 200);
System.out.println(res);

// long form of LengthFinder:
// (String a) -> { int len = a.length(); return len; }
```

Key points

- No-arg, one-arg and two-arg lambdas
- Return value of a lambda becomes the method's result
- Types of a, b are inferred from the interface

Output

```
Hello! Welcome
UCOE
32
300.0
```

Lambda Rule: Effectively Final Variables

```
int base = 10;
Calculator c = (a, b) -> a + b + base;    // OK: base never changes
System.out.println(c.add(1, 2));          // 13.0

// base = 20;     // ERROR if uncommented:
//                // local variables used in a lambda must be
//                // final or effectively final
```

Key points

- A lambda may read local variables of the enclosing method
- Those variables must not be changed after initialisation
- Fields (instance/static variables) can be changed freely

Method References - Shorthand for Lambdas

Lambda: str -> System.out.println(str)

==

Method reference: System.out::println

- A method reference calls the method on the RIGHT of :: from the body of the functional interface's only abstract method
- Use it when a lambda does nothing but call an existing method
- Parameters are passed along automatically - no need to repeat them
- Seen in our demos: .forEach(System.out::println) and MethodReferences::meth1

Four Types of Method References

Type	Syntax	Example
1. Static method	ClassName::staticMethod	MethodReferences::meth1
2. Instance method, particular object	object::instanceMethod	ob::meth2
3. Instance method, arbitrary object	ClassName::instanceMethod	String::toUpperCase
4. Constructor	ClassName::new	MyShow::new

- In the demo, ob1::play is really type 2 (a specific object ob1); type 3 looks like names.stream().map(String::toUpperCase)

MethodReferences: Setup

```
interface Printable { public void print(Object o); }       // functional interface

class MyShow {
    public MyShow() { }
    public MyShow(Object o) {
        System.out.println("MyShow Constructor " + o);
    }
    public void play(Object o) {
        System.out.println("Another Object instance method " + o);
    }
}

public class MethodReferences {
    public static void meth1(Object o) {                    // static method
        System.out.println("Static method " + o);
    }
    public void meth2(Object o) {                           // instance method
        System.out.println("Imstance method " + o);
    }
```

Key points

- Printable.print(Object) is the single abstract method
- Every method reference below must match its signature: one Object parameter
- MyShow(Object) is the constructor that Printable will call

MethodReferences: The Four Types in Action

```
// 1. static method
Printable p1 = MethodReferences::meth1;
p1.print("Welcome to method Reference");

// 2. instance method of a particular object
MethodReferences ob = new MethodReferences();
Printable p2 = ob::meth2;
p2.print(LocalDateTime.now());

// 3. instance method of object ob1
MyShow ob1 = new MyShow();
Printable p3 = ob1::play;
p3.print(Integer.valueOf(40));      // original: new Integer(40) is deprecated

// 4. constructor
Printable p4 = MyShow::new;
p4.print(LocalDate.now());
```

Key points

- print() simply forwards its argument to the referenced method
- MyShow::new calls the constructor that takes Object

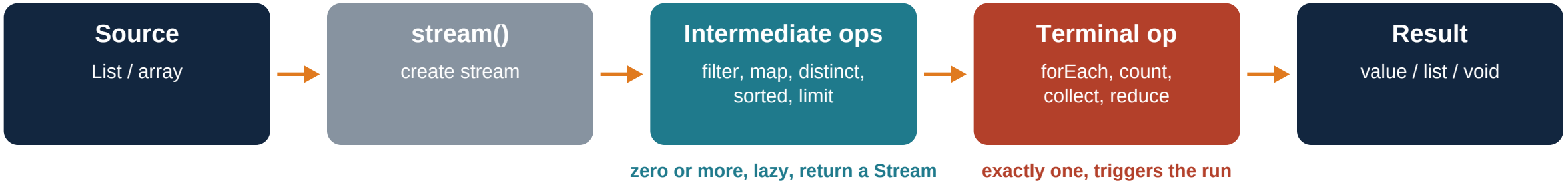
Output

```
Static method Welcome to method Reference
Imstance method <current date-time>
Another Object instance method 40
MyShow Constructor 2026-10-21
(dates show the day the demo runs)
```

Stream API - What is a Stream?

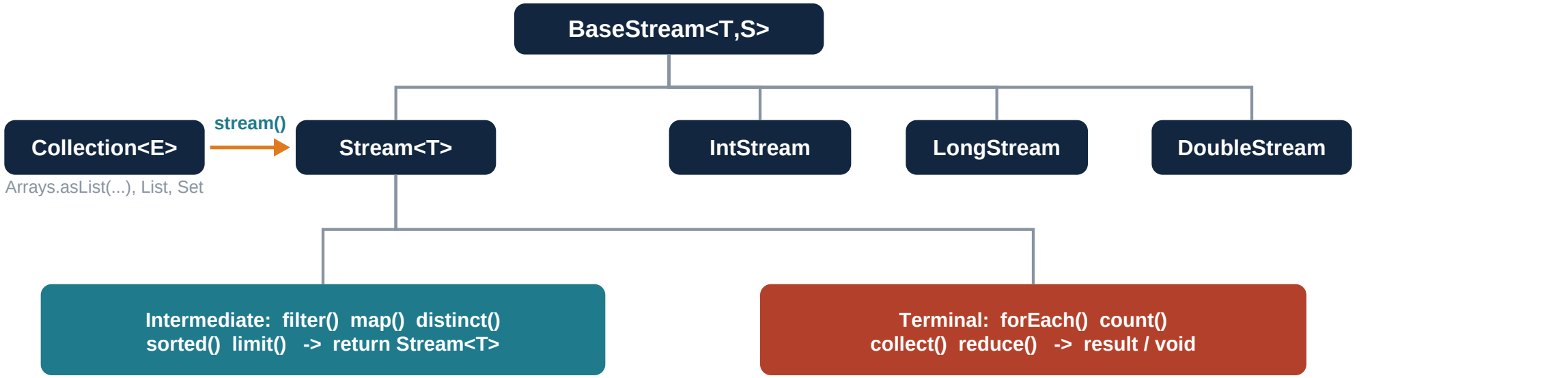
- A stream is a sequence of elements supporting aggregate operations (filter, map, count ...)
- It is NOT a data structure: it does not store elements, it carries them from a source
- It never modifies the source: list.stream().filter(...) leaves the list unchanged
- Operations are chained into a pipeline and run lazily - only when a terminal operation is called
- A stream can be consumed only ONCE (reuse = IllegalStateException)
- Sequential stream: elements are processed one by one (parallel streams exist too)
- Analogy: a factory conveyor belt - items pass through stations (filter, map) and a packing step at the end

Stream API - Pipeline Diagram




```
listint.stream()                       // 1. source -> stream
       .filter(a -> a > 10)            // 2. intermediate (Predicate)
       .distinct()                     // 2. intermediate
       .limit(3)                       // 2. intermediate
       .forEach(System.out::println);  // 3. terminal (Consumer)
```

Stream API - Class / Interface Diagram



Helper types used by the pipeline:

Collectors	Optional<T>	java.util.function
summingInt, toList, counting ...	result of reduce()	Predicate, Function, Consumer, BinaryOperator

Intermediate vs Terminal Operations

- Intermediate operations return a NEW stream and are lazy:
  - filter(Predicate), map(Function), distinct(), sorted(), limit(n)
- Terminal operations produce a result or side effect and START the pipeline:
  - forEach(Consumer), count(), collect(Collector), reduce(BinaryOperator)
- No terminal operation = nothing executes
- After a terminal operation the stream is closed; create a new one to reuse
- Chain style: list.stream().filter(...).map(...).forEach(...)
- Step style (as in the demos): Stream<Integer> s = list.stream(); Stream<Integer> f = s.filter(a -> a > 10);

Stream Operations Visualised (1/2)

filter: keeps elements that pass the Predicate (3 and 5 are dropped)



distinct: removes duplicates (the 2nd and 3rd 44 are dropped)



Grey = element removed. The original list is never changed.

Stream Operations Visualised (2/2)

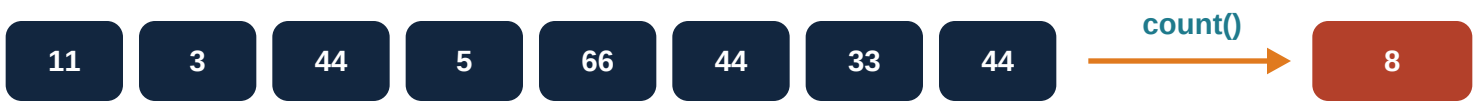
map: transforms every element using a Function (String -> int)



limit: keeps only the first n elements



count (terminal): number of elements, returned as a long



Demo1: forEach - Display the Contents

```
List<String> locations = Arrays.asList(
        new String[]{"Pune","Mumbai","Chennai","Banglore","Noida"});

// step by step
Stream<String> s1 = locations.stream();      // list -> stream
s1.forEach(str -> System.out.println(str));  // forEach takes a Consumer

// chained (same result)
locations.stream().forEach(str -> System.out.println(str));
```

Key points

- forEach is a TERMINAL operation
- Consumer.accept(T t) receives each element
- Each block prints the five locations once

Output

```
Pune
Mumbai
Chennai
Banglore
Noida
(printed twice: step-by-step and chained)
```

Demo2: map - Length of Every Location

```
Stream<String> s1 = locations.stream();

// map takes a Function:  R apply(T t)
Stream<Integer> intstream = s1.map(str -> str.length());
// "Pune" -> 4,  "Mumbai" -> 6 ... stored in Stream<Integer>

intstream.forEach(a -> System.out.println(a));

// chained
locations.stream().map(str -> str.length())
         .forEach(a -> System.out.println(a));
```

Key points

- map transforms each element into another value
- Type changes: Stream<String> becomes Stream<Integer>
- Same number of elements in and out

Output

```
4
6
7
8
5
```

Demo3: filter - Numbers Greater Than 10

```
List<Integer> listint = Arrays.asList(11, 3, 44, 5, 66, 44, 33, 44);

Stream<Integer> intstream = listint.stream();

// filter takes a Predicate:  boolean test(T t)
Stream<Integer> newintstream = intstream.filter(a -> a > 10);
newintstream.forEach(arg -> System.out.println(arg));

// chained
listint.stream().filter(arg -> arg > 10)
       .forEach(arg -> System.out.println(arg));
```

Key points

- Elements for which the Predicate is true are kept
- 3 and 5 are dropped; duplicates stay
- Output has fewer elements than the input

Output

```
11
44
66
44
33
44
```

Demo4: distinct - No Duplicates

```
Stream<Integer> intstream = listint.stream();

// distinct: element already seen earlier? then skip it
Stream<Integer> newintstream = intstream.distinct();
newintstream.forEach(arg -> System.out.println(arg));

// chained
listint.stream().distinct()
       .forEach(arg -> System.out.println(arg));

// listint = 11, 3, 44, 5, 66, 44, 33, 44
```

Key points

- Uses equals() to detect duplicates
- First occurrence is kept, later ones removed
- Order of the remaining elements is preserved

Output

```
11
3
44
5
66
33
```

Demo5: limit - First 3 Numbers

```
Stream<Integer> intstream = listint.stream();

// limit(n): take only the first n elements
Stream<Integer> newintstream = intstream.limit(3);
newintstream.forEach(arg -> System.out.println(arg));

// chained
listint.stream().limit(3)
       .forEach(arg -> System.out.println(arg));
```

Key points

- limit(3) stops after three elements
- Intermediate operation: returns a Stream

Output

```
11
3
44
```

Demo6: count - A Terminal Operation

```
Stream<Integer> intstream = listint.stream();

// count: terminating method, returns a long
long countofint = intstream.count();
System.out.println("Count " + countofint);

// chained
long c = listint.stream().count();
System.out.println("Count " + c);
```

Key points

- count() ends the pipeline
- Result type is long, not a Stream
- Counts duplicates too: 8 elements

Output

```
Count 8
Count 8
```

Demo7: collect with Collectors.summingInt

```
Stream<Integer> intstream = listint.stream();

// collect gathers the stream into a result using a Collector
int sumres = intstream.collect(Collectors.summingInt(i -> i));
System.out.println("Sum is " + sumres);

// chained
int sum = listint.stream().collect(Collectors.summingInt(i -> i));
System.out.println("Sum is " + sum);
```

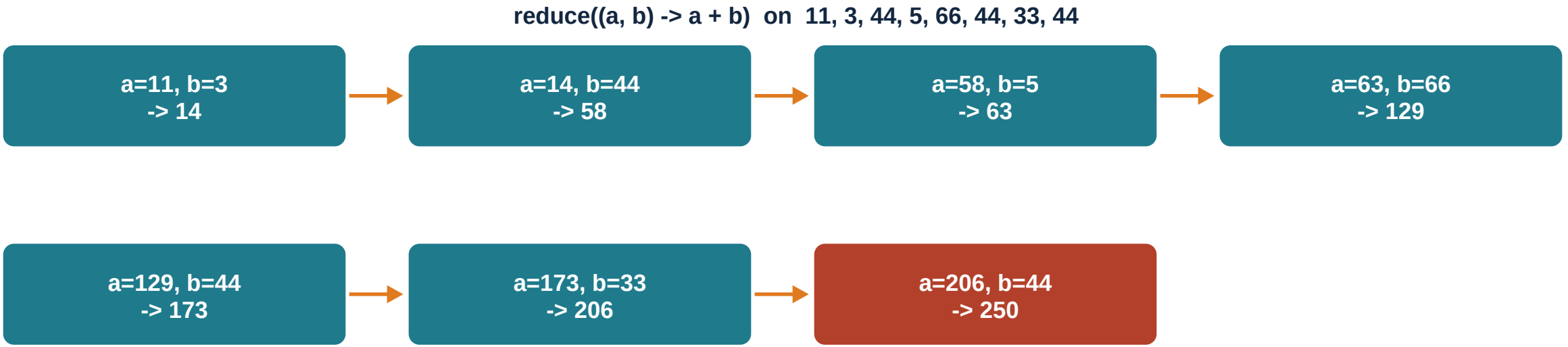
Key points

- collect is a TERMINAL operation
- Collectors.summingInt adds ints (i -> i unboxes)
- Other collectors: toList(), counting(), joining(), groupingBy()

Output

```
Sum is 250
Sum is 250
```

How reduce Works: Running Total



- a = running result so far, b = next element; the result becomes the next a
- 7 additions for 8 elements; final result 250 (wrapped in Optional)

Demo8: reduce - Sum Using BinaryOperator

```
Stream<Integer> intstream = listint.stream();

// reduce accepts a BinaryOperator:  T apply(T t, T u)
// it adds a and b repeatedly and returns an Optional<Integer>
Optional<Integer> intopres = intstream.reduce((a, b) -> a + b);
int sumres = intopres.get();
System.out.println("Sum is " + sumres);

// chained
Optional<Integer> opres = listint.stream().reduce((a, b) -> a + b);
int sum1 = opres.get();
System.out.println("Sum is" + sum1);
```

Key points

- reduce folds many elements into ONE value
- Returns Optional because the stream may be empty
- Note: the second println in the demo has no space: Sum is250

Output

```
Sum is 250
Sum is250
```

Optional - Result of reduce()

```
Optional<Integer> opt = listint.stream().reduce((a, b) -> a + b);

int sum = opt.get();            // 250
opt.isPresent();                // true
opt.orElse(0);                  // 250, or 0 if the stream was empty

// empty stream -> Optional.empty
Arrays.<Integer>asList().stream()
      .reduce((a, b) -> a + b)
      .orElse(0);               // 0   (get() would throw NoSuchElementException)
```

Key points

- Optional is a container that may or may not hold a value
- Safer than returning null
- Prefer orElse() / isPresent() to a blind get()

Loop vs Stream: Employees with Salary > 25000

```
// Java 7 style: external iteration (you write the loop)
for (Employee e : el) {
    if (e.getSalary() > 25000.00) {
        System.out.println(e);
    }
}

// Java 8 style: stream pipeline (internal iteration)
el.stream()
  .filter(e -> e.getSalary() > 25000.00)
  .forEach(e -> System.out.println(e));
```

Key points

- Same result, less code
- You describe WHAT (filter, print), the library decides HOW
- Easy to add more steps: sorted(), map(), limit() ...

EmployeeStreamDemo: filter on Objects

```
List<Employee> el = new ArrayList<Employee>();
el.add(new Employee(100, "Ram",     24000.00));
el.add(new Employee(105, "Arjun",   22000.00));
el.add(new Employee(109, "Sham",    28000.00));
el.add(new Employee(104, "Krishna", 27000.00));
el.add(new Employee(102, "Ganesh",  23000.00));

// sequential stream: elements are processed one by one
el.stream()
  .filter(e -> e.getSalary() > 25000.00)
  .forEach(e -> System.out.println(e));
```

Key points

- The Predicate reads the salary of each Employee
- Original order is kept: Sham, then Krishna
- The list el itself is not changed

Output

```
Employee [eid=109, name=Sham, salary=28000.0]
Employee [eid=104, name=Krishna, salary=27000.0]
```

EmployeeStreamDemo: sorted()

```
// DEMO 1: natural order - Employee.compareTo() (by eid)
el.stream()
  .sorted()
  .forEach(System.out::println);

// DEMO 2: Comparator + lambda (by name)
el.stream()
  .sorted(Comparator.comparing((Employee e) -> e.getName()))
  .forEach(System.out::println);

// DEMO 3: by salary (commented out in the demo)
el.stream()
  .sorted(Comparator.comparingDouble((Employee e) -> e.getSalary()))
  .forEach(System.out::println);
```

Key points

- sorted() needs Comparable, else ClassCastException
- Comparator.comparing replaces a whole comparator class
- The list order is unchanged: sorting happens inside the stream

Output

```
By eid   : 100 Ram, 102 Ganesh, 104 Krishna,
           105 Arjun, 109 Sham
By name  : Arjun, Ganesh, Krishna, Ram, Sham
By salary: Arjun 22000, Ganesh 23000, Ram 24000,
           Krishna 27000, Sham 28000
(each Employee prints in the usual toString format)
```

Stream Operations at a Glance

Operation	Type	What it does	Demo
forEach(Consumer)	terminal	does something with every element	Demo1
map(Function)	intermediate	transforms each element	Demo2
filter(Predicate)	intermediate	keeps matching elements	Demo3
distinct()	intermediate	removes duplicates	Demo4
limit(n)	intermediate	keeps the first n elements	Demo5
count()	terminal	number of elements	Demo6
collect(Collector)	terminal	gathers into a result	Demo7
reduce(BinaryOperator)	terminal	folds into one value	Demo8
sorted(...)	intermediate	orders the elements	Employee

Best Practices

- Keep lambdas short (one or two lines); extract longer logic into a method
- Use a method reference when the lambda only calls one method: System.out::println
- Prefer Comparator.comparing(...) to hand-written comparator classes
- Use streams for filter / map / reduce style work; keep a plain loop when it is clearer
- Do not modify the source collection while streaming over it
- Always finish the pipeline with a terminal operation
- Use orElse() / isPresent() with Optional instead of a blind get()
- Use Integer.valueOf(...) or autoboxing, not the deprecated new Integer(...)

Common Beginner Mistakes

- Using a lambda with an interface that has two abstract methods
- Changing a local variable that a lambda uses (must be effectively final)
- Forgetting the terminal operation: nothing prints, nothing runs
- Reusing a stream: IllegalStateException (stream has already been operated upon)
- Expecting filter/sorted to change the original list
- Confusing map (transform) with filter (select)
- Calling get() on an empty Optional: NoSuchElementException
- Using sorted() on objects that are not Comparable: ClassCastException

Practice Questions

- 1. What is a functional interface? Why can a lambda only target one?
- 2. Convert new SayHello(){ public void say(){...} } to a lambda.
- 3. Write lambdas for: no parameter, one parameter, two parameters.
- 4. Name the four kinds of method references with one example each.
- 5. Difference between intermediate and terminal operations? Give three of each.
- 6. Why does a stream pipeline without a terminal operation print nothing?
- 7. Using streams, print employees whose salary is below 25000, sorted by name.
- 8. Use map and reduce to find the total length of all locations.
- 9. Rewrite the Demo3 + Demo5 chain: numbers greater than 10, first two only.

Summary

- Lambda: (parameters) -> body, usable wherever a functional interface is expected
- Method reference: Class::method / object::method / Class::new as a lambda shortcut
- Stream: source -> intermediate ops (lazy) -> terminal op (starts the run)
- filter, map, distinct, sorted, limit return streams; forEach, count, collect, reduce end them
- reduce returns Optional; collect uses Collectors
- Streams do not change the source and can be used only once
- Result: shorter, more readable code than manual loops